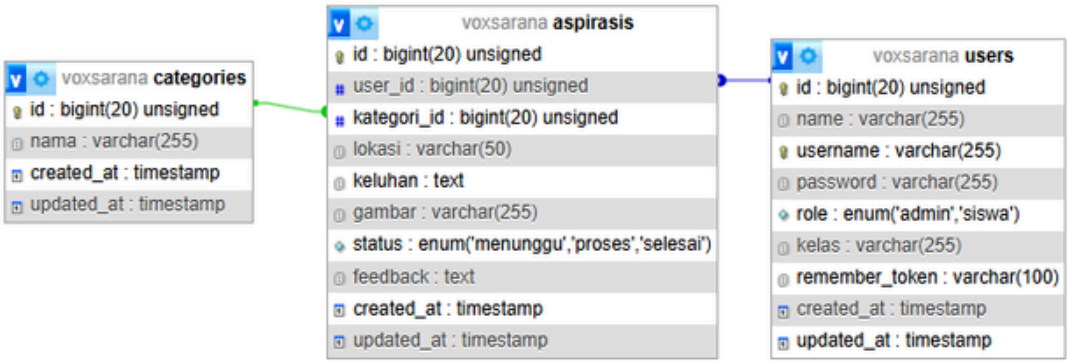
Dokumentasi UKK

Ahmad Sulfin Aqilla/01

Erd :



Deskripsi Program :

1. Tujuan Aplikasi

VoxSarana adalah platform web untuk mengelola pengaduan sarana dan prasarana sekolah.
Aplikasi ini memungkinkan:
- Siswa : Mengajukan pengaduan/aspirasi tentang kondisi fasilitas sekolah
- Admin : Mengelola pengaduan, kategori, dan memberikan feedback

2. Teknologi yang Digunakan

- Framework : Laravel 13.4.0
- Database : MySQL/MariaDB
- Frontend : Blade Template Engine, HTML, CSS, JavaScript
- Backend : PHP 8.4.2
- Authentication : Laravel Built-in Authentication
- File Storage : Local Storage dengan symbolic link

3. Fitur Utama

A. Authentication & Authorization

- Login : User dapat login dengan username dan password
- Register : Pembuatan akun baru dengan role siswa atau admin
- Logout : Session management yang aman dengan prevent back history
- Role-Based Access : Pembatasan akses berdasarkan role (siswa/admin)

B. Dashboard Siswa
- Membuat aspirasi baru dengan:
 - Pemilihan kategori
 - Input lokasi
 - Deskripsi keluhan
 - Upload gambar (optional, max 2MB)
- Melihat histori aspirasi pribadi:
 - Status pengaduan
 - Feedback dari admin
 - Gambar yang di-upload

C. Dashboard Admin
- Kelola Kategori :
 - Tambah kategori aspirasi baru
 - Lihat daftar kategori
 - Hapus kategori (jika tidak ada aspirasi menggunakannya)

- Lihat Semua Aspirasi :
 - Filter berdasarkan: Siswa, Kategori, Status, Rentang Tanggal
 - Lihat gambar yang di-upload
 - Update status aspirasi (Menunggu → Proses → Selesai)
 - Berikan feedback/respon ke siswa

D. Keamanan
- Prevent Back History : Mencegah akses kembali setelah logout
- Cache Control : Meta tags dan headers untuk prevent caching
- CSRF Protection : Token validation pada setiap request
- Session Management : Invalidate session saat logout
- Input Validation : Validasi data di server-side
- File Upload Security : Validasi tipe dan ukuran file

DOKUMENTASI FUNGSI/PROSEDUR :

1. Controller Functions
LoginController
Method: `showLoginForm()`
- Deskripsi : Menampilkan halaman login
- Route : GET `/login` → `login`
- Return : View `login.blade.php`
- Akses : Public (tidak perlu auth)

Method: `login(Request $request)`
- Deskripsi : Memproses login user
- Route : POST `/login`
- Validasi :
  - username: required
  - password: required
- Proses :
  1. Validasi input
  2. Cek kredensial di database
  3. Jika valid: buat session login
  4. Redirect ke dashboard sesuai role
- Return :
  - Sukses: Redirect ke siswa.dashboard atau admin.dashboard
  - Gagal: Kembali ke login dengan error

RegisterController

Method: `showRegistrationForm()`
- Deskripsi : Menampilkan halaman registrasi
- Route : GET `/register` → `register`
- Return : View `register.blade.php`

Method: `register(Request $request)`
- Deskripsi : Membuat user baru
- Route : POST `/register`
- Validasi :
  - name: required, min:2
  - username: required, unique
  - password: required, min:6
  - password_confirmation: required, same as password
  - kelas: required (untuk siswa)
- Proses :
  1. Validasi input
  2. Hash password
  3. Simpan user baru ke database
  4. Set role default = 'siswa'
- Return : Redirect ke login dengan success message

AspirasiController

Method: `siswaDashboard()`
- Deskripsi : Menampilkan dashboard siswa
- Route : GET `/siswa/dashboard` → `siswa.dashboard`
- Middleware : auth, role:siswa
- Proses :
  1. Ambil aspirasi user yang login
  2. Ambil semua kategori
  3. Return view dengan data
- Return : View `siswa/dashboard.blade.php`

Method: `store(Request $request)`
- Deskripsi : Menyimpan aspirasi baru
- Route : POST `/siswa/aspirasi` → `siswa.aspirasi.store`
- Middleware : auth, role:siswa
- Validasi :
  - kategori_id: required, exists di categories
  - lokasi: required, max:50
  - keluhan: required
  - gambar: optional, image, mimes:jpeg/png/jpg/gif, max:2048KB
- Proses :
  1. Validasi input
  2. Jika ada file gambar:
     - Generate nama: timestamp_namafile.ext
     - Simpan ke storage/app/public/aspirasis/
     - Simpan path ke database
  3. Create record di aspirasis table
- Return : Redirect ke siswa.dashboard dengan success message

Method: `adminDashboard(Request $request)`
- Deskripsi : Menampilkan dashboard admin dengan filter
- Route : GET `/admin/dashboard` → `admin.dashboard`
- Middleware : auth, role:admin
- Parameter Filter :
  - kategori_id: Filter by kategori
  - status: Filter by status (Menunggu/Proses/Selesai)
  - dari_tanggal: Filter by start date
  - sampai_tanggal: Filter by end date
  - search: Filter by nama siswa/username
- Proses :
  1. Query aspirasi dengan eager load (user, kategori)
  2. Apply filters yang ada
  3. Ambil semua kategori
  4. Return view dengan data
- Return : View `admin/dashboard.blade.php`

Method: `update(Request $request, Aspirasi $aspirasi)`
- Deskripsi : Update status dan feedback aspirasi
- Route : PUT `/admin/aspirasi/{aspirasi}` → `admin.aspirasi.update`
- Middleware : auth, role:admin
- Validasi :
 - status: required, in:Menunggu/Proses/Selesai
 - feedback: optional
- Proteksi : Jika status sudah 'Selesai', tidak bisa diubah lagi
- Proses :
 1. Cek apakah status selesai
 2. Validasi input
 3. Update status dan feedback
 4. Redirect dengan success message
- Return : Redirect ke admin.dashboard

CategoryController

Method: `store(Request $request)`
- Deskripsi : Tambah kategori aspirasi baru
- Route : POST `/admin/kategori` → `admin.kategori.store`
- Middleware : auth, role:admin
- Validasi :
 - nama: required, max:100, unique di categories table
- Proses :
 1. Validasi input
 2. Create category baru
 3. Redirect dengan success message
- Return : Redirect ke admin.dashboard

Method: `destroy(Category $category)`
- Deskripsi : Hapus kategori
- Route : DELETE `/admin/kategori/{category}` → `admin.kategori.destroy`
- Middleware : auth, role:admin
- Proteksi :
 - Cek apakah kategori masih digunakan di aspirasi
 - Jika ada: tampilkan error, tidak boleh dihapus
- Proses :
 1. Cek count aspirasi dengan kategori ini
 2. Jika > 0: error message
 3. Jika = 0: delete kategori
- Return : Redirect ke admin.dashboard

LogoutController

Method: `logout(Request $request)`
- Deskripsi : Logout user dan invalidate session
- Route : POST `/logout` → `logout`
- Proses :
1. Logout user (Auth::logout())
2. Invalidate session
3. Regenerate token
4. Redirect ke login
- Return : Redirect ke /login dengan success message

2. Model Functions

User Model
php
- Relasi
public function aspirasis() : HasMany
└─ return $this->hasMany(Aspirasi::class)

- Attributes/Methods yang available:
- id: ID user
- name: Nama user
- username: Username login
- password: Password terenkripsi
- role: 'siswa' atau 'admin'
- kelas: Kelas siswa
- created_at, updated_at: Timestamps

Aspirasi Model
php
- Relasi
public function user() : BelongsTo
└─ return $this->belongsTo(User::class)

public function kategori() : BelongsTo
└─ return $this->belongsTo(Category::class)

- Accessible Attributes:
- id: ID aspirasi
- user_id: FK ke users
- kategori_id: FK ke categories
- lokasi: Lokasi keluhan
- keluhan: Isi keluhan
- gambar: Path gambar
- status: Status keluhan
- feedback: Respon admin

Category Model
php
- Relasi
public function aspirasis() : HasMany
└─ return $this->hasMany(Aspirasi::class, 'kategori_id')

- Attributes:
- id: ID kategori
- nama: Nama kategori
- created_at, updated_at: Timestamps

3. Middleware Functions
RoleMiddleware
- Fungsi : Mengecek role user sebelum akses route
- Implementasi : Custom middleware
- Cara Kerja :
  php
 Route::middleware('role:admin')->group(...)
 Route::middleware('role:siswa')->group(...)
PreventBackHistory
- Fungsi : Prevent caching di level server
- Header yang dikirim :
 - Cache-Control: no-cache, no-store, max-age=0, must-revalidate
 - Pragma: no-cache
 - Expires: Sat, 26 Jul 1997 05:00:00 GMT

DEBUGGING :

1. Common Issues dan Solusinya
Issue 1: Column not found 'aspirasis.category_id'
Error:
SQLSTATE[42S22]: Column not found: 1054 Unknown column 'aspirasis.category_id'
Penyebab : Mismatch nama foreign key antara migration dan model relationship
Solusi :
php
- SALAH - default convention mencari category_id

```
public function aspirasis() {
  return $this->hasMany(Aspirasi::class);
}
```

- BENAR - specify nama foreign key yang sesuai

```
public function aspirasis() {
  return $this->hasMany(Aspirasi::class, 'kategori_id');
}
```

Prevention : Pastikan nama kolom foreign key di migration sama dengan yang dideklarasikan di relationship

Issue 2: Gambar tidak muncul setelah upload
Penyebab :
- Storage symbolic link belum dibuat
- Path gambar tidak benar di database
- Permission file tidak benar

Solusi :
bash
1. Create symbolic link

```
php artisan storage:link
```

2. Verify folder permission

```
ls -la storage/app/public/aspirasis/
```

3. Check yang disimpan di DB
SELECT gambar FROM aspirasis LIMIT 1;
Should show: aspirasis/timestamp_filename.jpg

4. Verify di view
```
<img src="{{ asset('storage/' . $aspirasi->gambar) }}">
```

Issue 3: Logout malfunction - user bisa klik back ke halaman sebelumnya
Penyebab : Prevent back history script tidak jalan atau cache browser aktif

Solusi :
html
```
<!-- Pastikan meta tags ada di <head> -->
<meta http-equiv="Cache-Control" content="no-cache, no-store, must-revalidate" />
<meta http-equiv="Pragma" content="no-cache" />
<meta http-equiv="Expires" content="0" />

<!-- Pastikan script di body sebelum </html> -->
<script>
 history.pushState(null, null, location.href);
 window.addEventListener('popstate', function() {
 history.pushState(null, null, location.href);
 });
</script>
```

Issue 4: Kategori tidak bisa dihapus dengan error generic
Penyebab : Kategori masih digunakan oleh aspirasi

Solusi :
php
```
- Cek apakah kategori masih digunakan
$category = Category::find($id);
if ($category->aspirasis()->count() > 0) {
 return redirect()->with('error', 'Kategori tidak bisa dihapus karena masih ada aspirasi');
}
```